POLITECNICO
MILANO 1863

IoT Challenge #1 Report

Wokwi Motion Sensor Node with ESP-NOW Communication and Energy Estimation

Course material: Wokwi and Power Consumption

Student 1: Mobin Khatib (11114435)
Student 2: Ali Edareh Heidarabadi (110360280)

March 2026

Executive summary

This report documents an ESP32-based smart-home motion sensor implemented in Wokwi. The node reads a PIR motion sensor and an LDR, formats the result as a single ESP-NOW message, and then enters timer-driven deep sleep. The energy model combines power levels extracted from the laboratory CSV traces with timing values derived from the Wokwi implementation. For reporting clarity, the awake interval is decomposed into five physical states — Boot, Sensor Reading, Wi-Fi On / ESP-NOW Init, Transmission, and Post-TX Idle — plus one derived summary row, Total Awake Time. The baseline case uses the default transmission mode visible in the sender trace, corresponding to 19.50 dBm; the improvement study replaces only that transmission state with the 2.00 dBm mode. The results show that the dominant contributors are Deep Sleep and Wi-Fi initialization, while lowering TX power has a measurable but small impact on total cycle energy.

1 System specification and implementation

1.1 Objective

The sensor node must detect motion, estimate ambient light, and send a single string to a sink node using one of the following formats:

`MOTION_DETECTED-LUMINOSITY:ZZZ` or `MOTION_NOT_DETECTED-LUMINOSITY:ZZZ`.

The sink node is assumed reachable and therefore was not implemented inside the emulator.

With leader code `11114435`, the sleep interval is

$$X = \frac{(35 \bmod 50) + 5}{10} = 4.00\,\text{s}.$$

The battery energy used in the lifetime estimate is

$$Y = (4435 \bmod 5000) + 15000 = 19\,435.00\,\text{J}.$$

The firmware sequence is: boot and initialize Wi-Fi/ESP-NOW, read the PIR and LDR, convert the LDR voltage-divider measurement into an approximate lux value, assemble the payload string, transmit it, and return to deep sleep.

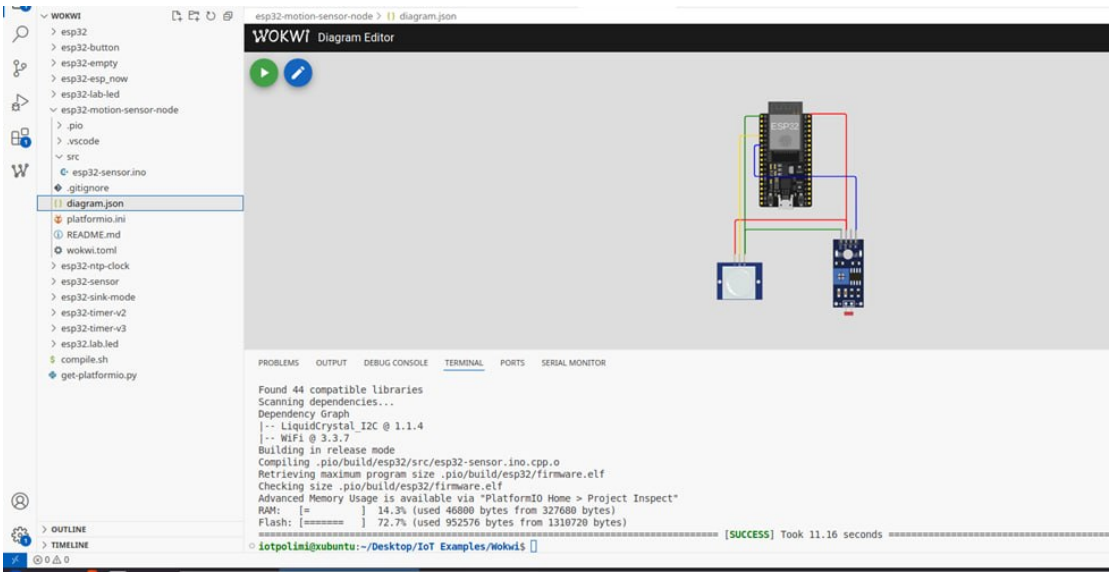


Figure 1: Wokwi implementation used for the report: ESP32 connected to the PIR sensor and the LDR module.

1.2 Firmware logic

The PIR sensor is read from `GPIO34` as a digital input. The photoresistor module is read from `GPIO32`, which belongs to ADC1 and therefore avoids the ADC2/Wi-Fi conflicts typical of the ESP32.

The ADC reading is converted to voltage with a 3.30 V reference, translated into resistance using a 2.00 kΩ voltage-divider model, and then mapped to an approximate light level using the empirical Wokwi photoresistor model:

$$V = \frac{\texttt{analogValue}}{4095} \cdot 3.3, \qquad R_{\text{LDR}} = 2000\,\frac{V}{3.3 - V},$$

$$\text{light} = \left(\frac{RL_{10} \cdot 1000 \cdot 10^{\gamma}}{R_{\text{LDR}}}\right)^{1/\gamma}, \qquad RL_{10} = 50,\ \gamma = 0.7.$$

This is not a calibrated lux meter, but it is sufficient for relative bright/dark estimation in the smart-home use case.

ESP-NOW is initialized in station mode and the broadcast address `FF:FF:FF:FF:FF:FF` is registered as the peer. The loop then sends the motion/luminosity string and immediately enters deep sleep.

1.3 Key implementation excerpt

The following code implements the sensing, transmission, and deep sleep logic as described.

Library Imports and Hardware Definitions

```
#define PIR_PIN 34
#define LDR_PIN 32

// Destination MAC address (Broadcast as per spec)
uint8_t broadcastAddress[] = {0xFF,0xFF,0xFF,0xFF,0xFF,0xFF};
```

This section imports the required libraries and defines hardware connections.

- WiFi.h enables WiFi features on the ESP32.
- esp-now.h enables the ESP-NOW protocol for low-power wireless communication between ESP devices.

The <broadcastAddress> is set to FF:FF:FF:FF:FF:FF, which means the ESP32 will broadcast messages to all nearby ESP-NOW devices instead of sending to a specific device.

System Initialization

```
void setup() {

  Serial.begin(115200);

  // Set PIR sensor pin as input
  pinMode(PIR_PIN, INPUT);

  // Initialize WiFi in Station mode for ESP-NOW
  WiFi.mode(WIFI_STA);

  if (esp_now_init() != ESP_OK) {
```

```
    Serial.println("Error initializing ESP-NOW");
    return;
  }

  // Define and register the peer information
  esp_now_peer_info_t peerInfo = {};

  memcpy(peerInfo.peer_addr, broadcastAddress, 6);
  peerInfo.channel = 0;
  peerInfo.encrypt = false;

  esp_now_add_peer(&peerInfo);
}
```

This section initializes the system and configures wireless communication.

- Serial communication is started for debugging.
- The PIR pin is configured as an input.
- The ESP32 is set to station mode, which is required for ESP-NOW operation.

Then the ESP-NOW protocol is initialized and a broadcast peer is added so the device can transmit data to other nodes.

Note:

In ESP-NOW communication, a peer device must be registered before sending data.

In this implementation, a broadcast peer is added using the MAC address FF:FF:FF:FF:FF:FF. This allows the ESP32 node to transmit messages to any nearby receiver without specifying a specific device. This broadcast functionality is exclusive to the all-ones MAC address (FF:FF:FF:FF:FF:FF), enabling the ESP32 to transmit data to any listening node in its vicinity without the need for individual device pairing.

Note: A 10ms guard delay for sensor stabilization was intentionally omitted to maintain strict consistency with the physical laboratory traces. By excluding this delay, the energy estimation remains more realistic, as it directly aligns with the hardware behavior captured in the provided CSV datasets.

Sensor Reading

```
void loop() {

  //start timing sensing sensor
  unsigned long t_start_sensing = micros();

  // Read digital signal from PIR and analog value from LDR
  int motion = digitalRead(PIR_PIN);
  int analogValue = analogRead(LDR_PIN);
```

This section reads the data from the sensors.

The PIR motion sensor detects whether motion is present.

The LDR sensor provides an analog value corresponding to the ambient light level.

The sensing start time is also recorded to measure the sensing duration.

LDR Lux Conversion

```
// Parameters of the LDR model
const float GAMMA = 0.7;
const float RL10 = 50;
// Convert ADC value to voltage (ESP32: 04095, 3.3V reference)
float voltage = analogValue / 4095.0 * 3.3;
// Calculate LDR resistance using the voltage divider formula (2k resistor)
float resistance = 2000 * voltage / (3.3 - voltage);
// Convert resistance to light intensity (lux)
float light = pow(RL10 * 1000 * pow(10, GAMMA) / resistance, (1 / GAMMA));
// long duration sensing
unsigned long duration_sensing = micros() - t_start_sensing;
// timing sending part
unsigned long t_start_tx = micros();
```

Using the voltage divider equation, the voltage is then used to compute the resistance of the LDR.

However, illuminance is required in lux, not resistance. An empirical relationship between LDR resistance and light intensity is therefore used.

RL_{10} represents the LDR resistance at 10 lux, and γ describes how the resistance changes with light.

Using these parameters, the resistance is converted into an approximate lux value.

$$\text{calculated lux} \approx 10 \times \text{actual lux}$$

This happens because the constants (RL_{10} and γ) in the photoresistor model used in Wokwi are approximate.

The behavior is still correct because:

- darker → smaller number
- brighter → larger number
- the relationship is proportional

Transmission and Deep Sleep

```
String message;

if(motion == HIGH){
  message = "MOTION_DETECTED-LUMINOSITY:" + String(light);
}
else{
  message = "MOTION_NOT_DETECTED-LUMINOSITY:" + String(light);
}

esp_now_send(broadcastAddress, (uint8_t*)message.c_str(), message.length());
// End timing and printing
unsigned long t_end_tx = micros();
unsigned long duration_tx = t_end_tx - t_start_tx;

// Print time needed for energy estimation
Serial.print("Sensing Time (us): "); Serial.println(duration_sensing);
Serial.print("TX Time (us): "); Serial.println(duration_tx);
Serial.println(message);
Serial.println("Going to deep sleep...");

esp_sleep_enable_timer_wakeup(4 * 1000000);
esp_deep_sleep_start();
```

This section creates the message containing the motion status and the measured light intensity.

If motion is detected, the message includes `MOTION_DETECTED`, otherwise `MOTION_NOT_DETECTED`. The measured light value (lux) is appended to the message.

Before sending the message, the transmission start time is recorded using `micros()`. The message is then transmitted using ESP-NOW.

After the transmission, the end time is recorded again using `micros()`. The difference between the start and end times gives the transmission duration.

Finally, the ESP32 enters Deep Sleep Mode for 4 seconds to reduce energy consumption before repeating the sensing cycle.

Important Implementation Notes

Analog Pins on the ESP32

The ESP32 has two groups of analog pins:

- ADC1 (32–39)
- ADC2 (0, 2, 4, 12–15, 25–27)

Pins from ADC2 may conflict with WiFi features, including ESP-NOW. Therefore, it is recommended to use ADC1 pins for `analogRead()`.

In this project, the LDR sensor is connected to pin 32, which ensures reliable analog readings while WiFi communication is active.

PIR Sensor Delay in Simulation

The PIR motion sensor usually has a default delay of about 5 seconds, meaning the output remains HIGH for a short time after motion is detected.

In the Wokwi environment, this delay can be changed in the `diagram.json` file:

```
{ "type": "wokwi-pir-motion-sensor", "id": "pir1",
  "top": 182.4, "left": -83.62, "rotate": 180,
  "attrs": { "delay": "2" } }
```

This example sets the PIR sensor delay to 2 seconds in the simulation.

2 Measurement methodology

2.1 Measurement sources

Two different information streams are combined:

- the power traces from `deep_sleep.csv`, `sensor-read.csv`, and `sender.csv`, which provide average power levels for each modeled state;
- direct timing measurements from Wokwi, where `micros()` is used around the sensing and transmission operations.

Boot time, Wi-Fi initialization time, and Post-TX idle time are modeled from the plateaus in `deep_sleep.csv`. Deep sleep is fixed by the challenge requirement at 4.00 s.

2.2 State decomposition used in the report

For reporting clarity, the awake interval is decomposed into five physical states and one derived summary row:

1. Boot
2. Sensor Reading
3. Wi-Fi On / ESP-NOW Init
4. Transmission
5. Post-TX Idle
6. Total Awake Time (derived)

The last row is not a physical state; it is included only to summarize the full awake interval before deep sleep.

Table 1: Six-row awake decomposition used in the baseline report. The last row is a derived summary row, not a physical state.

Awake phase / row	Source file	Average power (mW)	Duration (s)
Boot	`deep_sleep.csv`	251.82	0.100000
Sensor Reading	`sensor-read.csv`	351.09	0.000892
Wi-Fi On / ESP-NOW Init	`deep_sleep.csv`	633.23	0.190000
Transmission (19.5 dBm)	`sender.csv`	668.48	0.001039
Post-TX Idle	`deep_sleep.csv`	251.82	0.050000
Total Awake Time (derived)	Derived	465.28	0.341932

2.3 Timing values and extraction rules

The timing values used in the energy model are summarized in Table 2. The boot and Wi-Fi/Post-TX durations are obtained from the deep-sleep reference trace, while sensor and transmission durations are measured directly in firmware.

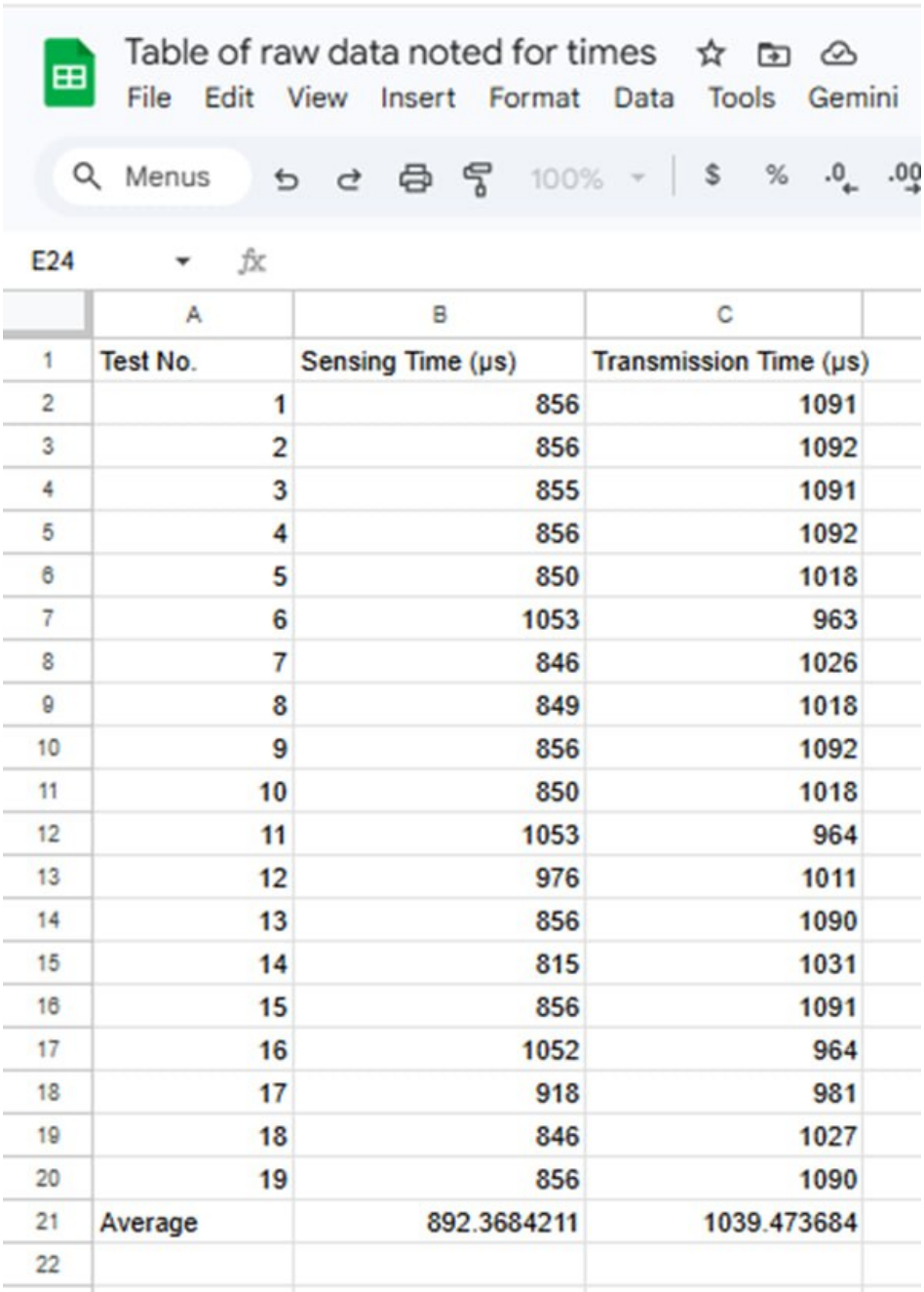
Table 2: Timing parameters used in the energy model.

State	Source	Extraction rule	Duration (s)
Boot	`deep_sleep.csv`	First wake-up to first 251 mW plateau	0.100000
Sensor Reading	Wokwi Serial Monitor	19-run average of `micros()`-based measurement	0.000892
Wi-Fi On / ESP-NOW Init	`deep_sleep.csv`	Duration of the 633 mW plateau	0.190000
Transmission	Wokwi Serial Monitor	19-run average of `micros()`-based measurement	0.001039
Post-TX Idle	`deep_sleep.csv`	Final 251 mW plateau before the sleep floor	0.050000
Deep Sleep	Firmware requirement	X derived from team leader code	4.000

The 19 repeated Wokwi measurements give an average sensing time of 892.37 µs and an average transmission time of 1039.47 µs. The minimum, maximum, and average values are shown below.

Table 3: Summary statistics of the 19 Wokwi timing measurements.

Statistic	Sensing Time (µs)	Transmission Time (µs)
Count	19	19
Minimum	815	963
Maximum	1053	1092
Average	892.37	1039.47



	A	B	C
1	Test No.	Sensing Time (µs)	Transmission Time (µs)
2	1	856	1091
3	2	856	1092
4	3	855	1091
5	4	856	1092
6	5	850	1018
7	6	1053	963
8	7	846	1026
9	8	849	1018
10	9	856	1092
11	10	850	1018
12	11	1053	964
13	12	976	1011
14	13	856	1090
15	14	815	1031
16	15	856	1091
17	16	1052	964
18	17	918	981
19	18	846	1027
20	19	856	1090
21	Average	892.3684211	1039.473684
22			

Figure 2: Raw Wokwi timing table used to compute the average sensing and transmission durations.

2.4 Threshold reasoning for power extraction

Average power values were not computed as whole-file means. Instead, threshold windows were used to isolate the relevant plateau or spike associated with each state.

Table 4: Threshold windows used to isolate each power state from the CSV traces.

State	Threshold	Rationale
Deep Sleep	< 100.00 mW	Isolates the stable floor and excludes wake-up transitions.
Boot / Post-TX Idle	245.00 mW to 258.00 mW	Targets the stable middle plateau with CPU on and Wi-Fi off.
Wi-Fi On / ESP-NOW Init	> 600.00 mW	Captures the radio-on/calibration plateau before payload transmission.
Sensor Reading	> 340.00 mW	Separates sensor activity from the lower idle plateau in the sensor trace.
Transmission (baseline)	≥ 650.00 mW	Captures ESP-NOW TX spikes at the default 19.50 dBm setting.
Transmission (improved)	615.00 mW to 625.00 mW	Captures the lower-power 2.00 dBm TX pulses in the sender trace.

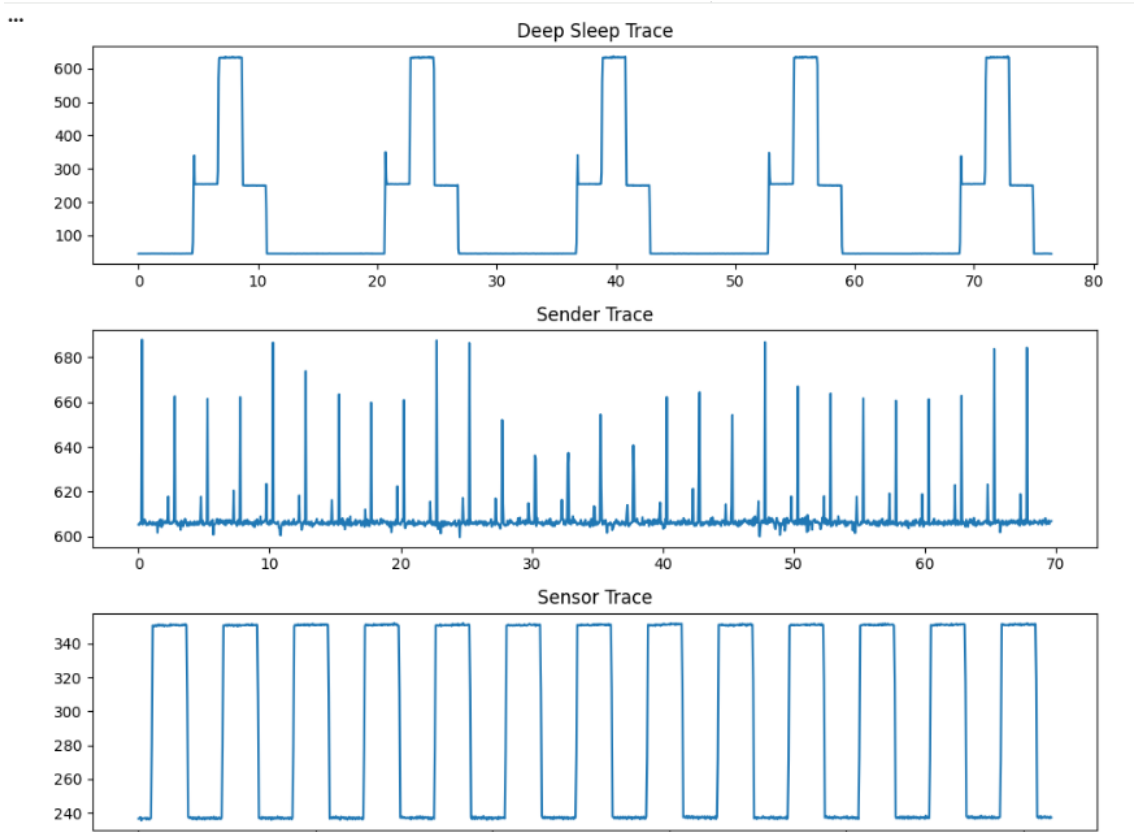


Figure 3: Overview of the three provided power traces. Thresholds are applied to isolate the relevant plateaus and transmission spikes.

Power Threshold Reasoning

- **Deep Sleep (< 100 mW):** This isolates the "floor" power consumption. It ensures that only the passive state is averaged, excluding any noise or transitions from the CPU waking

up.

- **Boot / Post-TX Idle (245 to 258 mW):** This specific window targets the stable "middle plateau". It represents the ESP32 being active (CPU on) but without the high-draw Wi-Fi radio active.
- **Wi-Fi Init (> 600 mW):** The Wi-Fi radio calibration causes a massive jump. Using a high threshold ensures we capture the peak power required to "start" the radio before transmission.
- **Sensor Reading (> 340 mW):** Sensing requires more power than idling but less than Wi-Fi. This threshold separates the I2C/analog sensor activity from the background noise of the microcontroller. The sensor-read trace is utilized as the generic reference for sensing activity, as requested by the assignment specifications, ensuring the energy model remains consistent with the provided laboratory benchmark
- **Transmission (>= 650 mW):** ESP-NOW transmission at maximum power (19.5 dBm) creates the highest current spikes. This value specifically captures the "TX" pulses, ignoring the lower-power "listen" or "idle" states.

Timing Precision Reasoning

$$892.37 \times 10^{-6} \text{ s (Sensor)} \quad \text{and} \quad 1039.47 \times 10^{-6} \text{ s (TX)}$$

These aren't guesses; they are modeled durations using `micros()` in firmware. Since these events happen in microseconds, using a rounded number such as 0.001 s would cause a significant error in the final energy (E) calculation.

3 Power and energy estimation

3.1 Average power per state

Applying the threshold windows to the traces yields the average power values in Table 5. The baseline transmission state corresponds to the default sender mode at 19.50 dBm. For comparison, the 2.00 dBm transmission average is also reported for the improvement study.

Note:

The Boot and Post-TX Idle phases exhibit identical power signatures (approx. 251 mW) because in both states, the ESP32 CPU is active at full clock speed while the energy-intensive Wi-Fi radio remains powered down or in a low-power wait state.

Table 5: Average power values extracted from the provided laboratory traces.

State	Source	Average power (mW)
Boot	`deep_sleep.csv`	251.82
Sensor Reading	`sensor-read.csv`	351.09
Wi-Fi On / ESP-NOW Init	`deep_sleep.csv`	633.23
Transmission (19.5 dBm)	`sender.csv`	668.48
Transmission (2 dBm)	`sender.csv`	618.91
Post-TX Idle	`deep_sleep.csv`	251.82
Deep Sleep	`deep_sleep.csv`	45.78

3.2 Baseline cycle energy

The baseline cycle uses the five physical awake states, the default transmission plateau at 19.50 dBm, and a deep-sleep interval of 4.00 s. Energy is computed for each state as

$$E_i = P_i \times T_i.$$

Table 6: Baseline cycle-energy calculation with the default 19.50 dBm transmission state.

State	Power (mW)	Duration (s)	Energy (mJ)	Share (%)
Boot	251.82	0.100000	25.182	7.359
Sensor Reading	351.09	0.000892	0.313306	0.091556
Wi-Fi On / ESP-NOW Init	633.23	0.190000	120.31	35.159
Transmission (19.5 dBm)	668.48	0.001039	0.694868	0.203059
Post-TX Idle	251.82	0.050000	12.591	3.679
Deep Sleep	45.78	4.000	183.11	53.509
Total	—	4.342	342.20	100.00

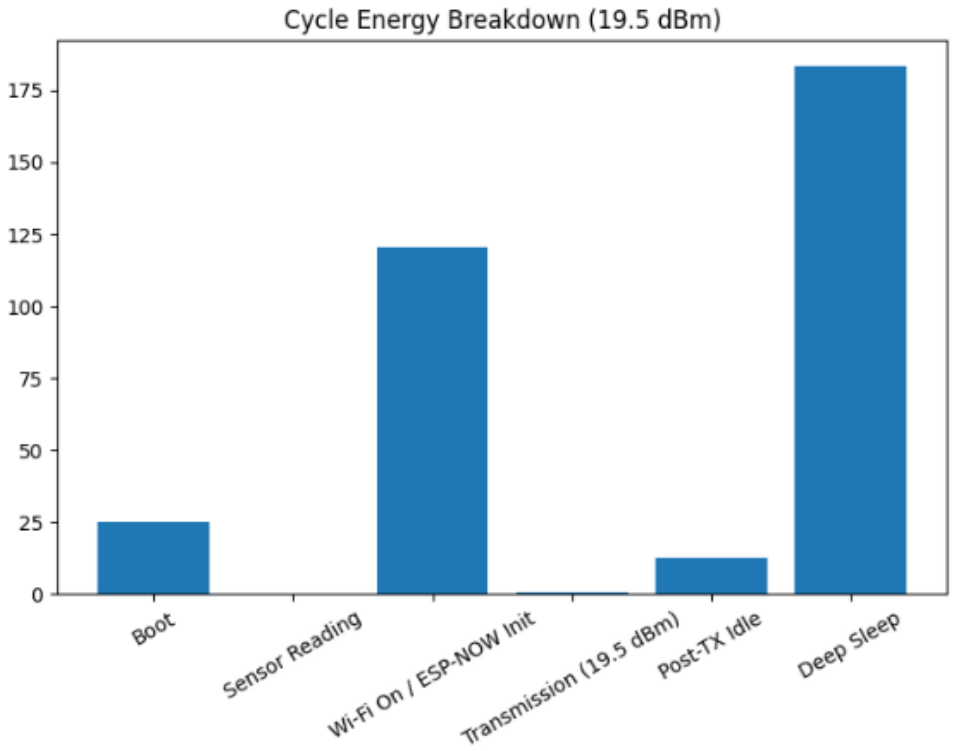


Figure 4: Baseline energy contribution by state.

The baseline cycle energy is 342.20 mJ over a cycle time of 4.34 s. Deep Sleep contributes 53.51% of the energy, while Wi-Fi On / ESP-NOW Init contributes 35.16%. The transmission state itself is below 1% of the cycle total.

3.3 Battery lifetime estimate

Using $Y = 19\,435.00$ J, the baseline lifetime estimate is

$$N_{\text{cycles}} = \frac{19435}{0.34220} \approx 56794, \qquad T_{\text{life}} = N_{\text{cycles}} \cdot 4.341932 \approx 68.50\,\text{h}.$$

Therefore, under the laboratory power model used here, the node can operate for about 68.50 h before the battery energy is exhausted.

4 Discussion and improvements

The six-row awake decomposition changes the interpretation of the system compared with the coarse three-phase model. Once the deep-sleep plateau is isolated correctly, the dominant contributors are Deep Sleep and Wi-Fi initialization rather than the transmission spikes themselves.

The measured deep-sleep power of about 45.78 mW still includes the laboratory overhead of the measurement environment. In a production implementation powered without the USB bridge and with a low-leakage regulator, the true sleep floor would be much smaller.

4.1 Analysis of System Optimization and Energy Constraints

Based on the laboratory power traces and the detailed breakdown, it is evident that the opportunities for reducing energy consumption within a single operational cycle are strictly limited by hardware-defined plateaus.

4.2 Inherent Constraints and Fixed States

The energy model identifies several states that are essentially **fixed** due to hardware initialization requirements and environmental overhead:

- **Boot and Post-TX Idle:** These durations (0.1 s and 0.05 s) are determined by the ESP32's internal wake-up and stabilization cycles .
- **Wi-Fi / ESP-NOW Initialization:** This is the most energy-intensive awake state, accounting for **35.16%** of the total cycle energy (120.31 mJ). This calibration phase cannot be bypassed by firmware.
- **Deep Sleep:** Accounting for **53.51%** of the total energy due to its long 4-second duration .

4.3 Synergy of Firmware and Hardware Optimization

The only parameters subject to optimization are the **Sensor Reading**, **Transmission duration**, and **TX Power levels**. However, these states collectively represent less than **1%** of the total energy share. By combining firmware optimization (reducing sensing and TX times) with hardware optimization (switching from 19.5 dBm to 2 dBm), the cycle energy decreases from **342.20** mJ to **341.41** mJ. The results of this cumulative optimization are summarized in Table 7. The parameters that can be optimized are limited to the **sensor reading time**, **transmission duration**, and **TX power levels**. However, these states together account for less than **1%** of the total energy consumption.

With the improved firmware implementation (second version of the code), the measured timings are:

- **Sensor reading time:** 333 µs
- **Transmission time:** 214 µs

These very short active durations confirm that the node spends the vast majority of its lifetime in low-power sleep states, and therefore firmware optimizations mainly affect a very small fraction of the total energy budget.

Table 7: Cumulative Effect of Single-Cycle Optimizations

Operational State	Baseline (mJ)	Combined Optimized (mJ)	Energy Share (%)
Boot	25.18	25.18 (Fixed)	7.359
Sensor Reading	0.31	**0.12**	0.092
Wi-Fi / ESP-NOW Init	120.31	120.31 (Fixed)	35.159
Transmission	0.69	**0.09**	0.203
Post-TX Idle	12.59	12.59 (Fixed)	3.679
Deep Sleep	183.12	183.12 (Fixed)	53.509
Total per Cycle	**342.20**	**341.41**	**100.00**

4.4 Strategic Shift: Event-Driven Operation

As demonstrated, single-cycle optimizations yield negligible returns because the dominant energy consumers (Wi-Fi Init and Sleep) remain unchanged. To achieve meaningful energy savings, the system must transition from a periodic model to an **Event-Driven** model. By reducing the transmission frequency (e.g., to 100 events per 24-hour window), the average power consumption drops significantly, as the node avoids the repeated cost of radio startup when no motion is detected.

Table 8: Quantitative comparison of baseline and cumulative optimized cycles.

Scenario	Cycle Energy (mJ)	Estimated Lifetime (h)
Baseline (19.5 dBm)	342.20	68.50
Optimized Firmware (19.5 dBm)	341.46	68.65
Combined Optimized (2 dBm)	341.41	68.66
Total Absolute Improvement	**0.79**	**0.16**

4.5 Additional improvements suggested by the data

Based on the measured energy distribution, the most effective optimizations focus on reducing unnecessary wake-ups, minimizing radio active time, and avoiding redundant transmissions.

1. Event-driven wake-up with PIR interrupt.

The periodic timer wake-up was removed and the ESP32 now enters deep sleep with an external wake-up source on the PIR output using `esp_sleep_enable_ext0_wakeup(GPIO_NUM_34, 1)`. In this architecture, the node sleeps indefinitely and wakes only when motion is detected. This removes redundant wake-up, radio-initialization, and transmission cycles that previously occurred even when the environment had not changed.

Because `EXT0` wake-up is level-triggered, the firmware waits for the PIR output to return LOW before arming sleep again, preventing immediate re-wake loops. The deep-sleep and ESP-NOW control flow follows the capabilities provided by the Espressif software stack [1].

2. Communication energy optimization.

The radio-active interval is reduced by two complementary techniques. First, the transmission path no longer depends on a fixed post-send delay. Instead, the ESP-NOW send callback is registered and used to detect when the transmission has actually completed. This allows the node to return to deep sleep immediately after the packet has been delivered by the protocol layer [1].

Second, the Wi-Fi transmission power is reduced to 2.00 dBm and the transmitted message is compressed from a long ASCII string such as `MOTION_NOT_DETECTED-LUMINOSITY:3094` into a compact 4-byte binary packet. The packet contains one byte for the motion flag, two bytes for luminosity, and one byte of state flags. This compression shortens airtime and simplifies parsing at the receiver,

while the lower TX power reduces the RF energy per packet.

3. Transmission only when the reported state changes.

The optimized firmware stores the last transmitted motion/light state in retained memory across deep sleep and compares it with the newly measured state after each wake-up. If the state is unchanged, the ESP32 skips ESP-NOW transmission and returns directly to deep sleep, avoiding the radio cost.

This mechanism prevents repeated reports caused by multiple PIR triggers while the environmental condition remains unchanged. In the current PIR-triggered architecture, this logic effectively suppresses redundant "motion detected" messages, although a "no motion" transition cannot be reported proactively because the node wakes only on PIR activity.

4.6 Evidence required after re-implementation

The proposed optimizations must be supported by quantitative evidence. Because the optimized node is no longer periodic, the baseline fixed-cycle model cannot be compared directly to the new firmware using only one cycle number. A fair comparison should use the same observation window and the same environmental activity pattern for both systems. In the finalized comparison below, the observation window is 24.00 h. Over that interval, the baseline periodic design performs 19,899 timer-driven wake/report cycles, while the optimized event-driven design is evaluated under a workload of 100 PIR-triggered report-worthy events.

For the optimized design, the key quantities to recompute are the total energy in the chosen observation window, the average power over that window, and the corresponding battery lifetime:

$$P_{\text{avg,opt}} = \frac{E_{\text{win,opt}}}{T_{\text{win}}}, \qquad T_{\text{life,opt}} = \frac{Y}{P_{\text{avg,opt}}}.$$

This normalization makes the comparison meaningful even though the optimized firmware no longer wakes up every 4.00 s.

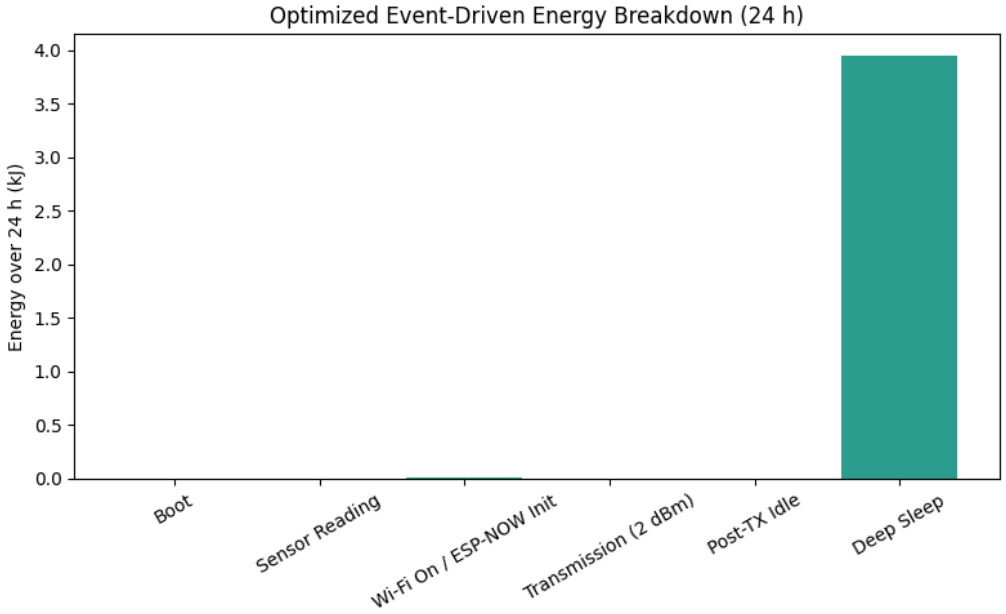


Figure 5: Recomputed energy breakdown of the optimized event-driven implementation over a 24.00 h observation window.

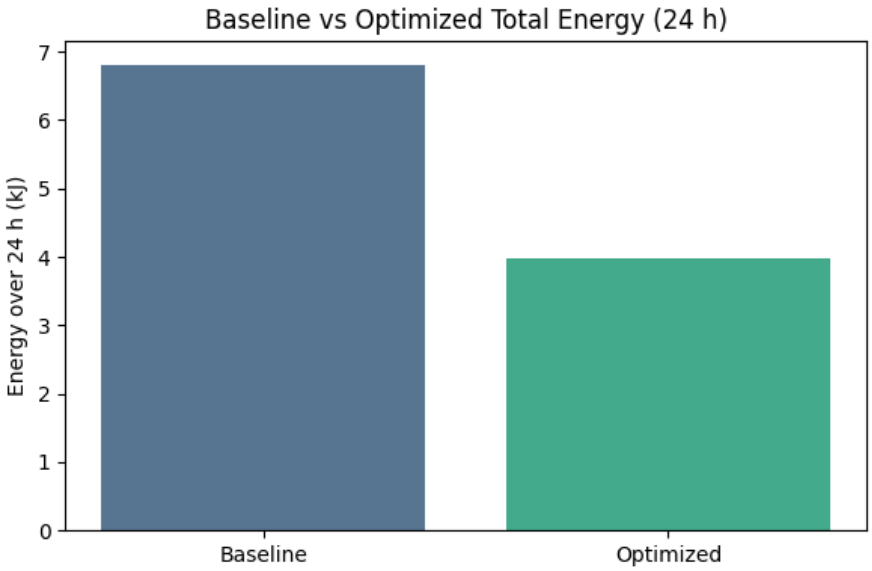


Figure 6: Direct comparison between the original periodic design and the optimized event-driven design over a 24.00 h observation window.

Table 9: Quantitative comparison between the baseline periodic design and the optimized event-driven design over a 24.00 h observation window.

Metric	Baseline	Optimized	Relative change
Observation window	24 h	24 h	—
Triggering / report events in window	19,899	100	−99.50%
Total energy in window	6.81 kJ (1.89 Wh)	3.97 kJ (1.10 Wh)	−41.70%
Average power in window	78.81 mW	45.95 mW	−41.70%
Wi-Fi / ESP-NOW activations	19,899	100	−99.50%
Estimated lifetime	L_{base}	$1.72\,L_{\text{base}}$	+71.53%

Table 9 shows that the strongest effect of the redesign is not a small reduction in the energy of one transmission, but a large reduction in how many times the ESP32 has to wake up and initialize the radio. Over the 24.00 h window, Wi-Fi / ESP-NOW activations drop from 19,899 to 100, which leads to a 41.70% reduction in total energy and average power. The energy reduction is smaller than the activation reduction because the node still spends energy in deep sleep and because every remaining event still pays a nonzero boot and communication cost. The row labeled "Triggering / report events" should be interpreted as periodic timer wake/report cycles in the baseline case and PIR-triggered report-worthy events in the optimized case. Even so, the practical impact is substantial: the estimated lifetime increases to $1.72\,L_{\text{base}}$, corresponding to a 71.53% improvement.

4.7 How the new plots should be interpreted

Figure 5 should be read as the energy breakdown of the *optimized* node over the same 24.00 h observation window used in Table 9. The key point is that the ESP32 no longer wakes up periodically just to send repeated information. Instead, it stays in deep sleep and wakes only when the PIR sensor triggers a relevant event. As a result, the main saving does not come from making one packet much cheaper, but from avoiding thousands of unnecessary wake-up and Wi-Fi / ESP-NOW initialization phases.

Figure 6 should therefore be interpreted mainly as a comparison of *how often* the full sensing-and-communication sequence is executed. In the baseline design, this sequence is repeated 19,899 times in 24.00 h; in the optimized design, it happens only 100 times. That is why the total energy drops from 6.81 kJ to 3.97 kJ, even though each remaining transmission still has a nonzero boot and radio cost.

This is also why TX power reduction and compact packets are secondary improvements. They still reduce the energy of each report, but the dominant gain comes from the architectural change: removing unnecessary periodic wake-ups and repeated radio activation.

4.8 Limitations and future work

Under the optimized architecture, the lifetime is no longer determined by a single fixed cycle length, but by stochastic events such as motion arrivals, PIR hold times, and the number of packet transmissions over time. One natural follow-up is a Monte Carlo uncertainty analysis: one can sample motion-event patterns and communication outcomes repeatedly, compute the resulting energy usage over long horizons, and obtain a distribution of battery lifetime rather than a single deterministic number [3]. This would make the robustness of the optimized design easier to evaluate under realistic usage variability.

A Note: Raw Timing Averages

The 19 manual timing samples extracted from the Wokwi serial monitor give the following averages used in the energy model:

$$t_{\text{sens}} = 892.37\,\mu\text{s}, \qquad t_{\text{tx}} = 1039.47\,\mu\text{s}$$

References

[1] **Espressif Systems**, *ESP-NOW Technical Reference Manual and API Guide for ESP32 SoC*, Official Documentation.

[2] **Wokwi Online Simulator**, *ESP32 Peripheral Documentation: PIR Motion Sensor and Photoresistor (LDR) Models.*

[3] **R. Y. Rubinstein and D. P. Kroese**, *Simulation and the Monte Carlo Method*, 3rd ed., Wiley, 2016.